

GPU-Accelerated Image Compression with Content- Aware Quality Mapping

Parallel & Distributed Computing Project

Team:

- 1) Aditya Chauhan - 102303130
- 2) Gaurang Garg - 102303134
- 3) Saksham Mittal - 102303135



What is JPEG?

JPEG is fundamentally a compression **algorithm**, not a file format itself. The actual file formats such as JFIF and EXIF are merely wrappers that encapsulate the compressed data.



Lossy Compression

Data is intentionally discarded to reduce file size, unlike PNG or BMP which preserve all information



Perceptual Optimization

Exploits human visual system limitations to discard imperceptible information

Two Key Perceptual Limitations

01

Color Sensitivity

Humans are significantly less sensitive to fine color detail than to brightness variations

02

Frequency Perception

We struggle to perceive high-frequency intensity changes in images.
We are physically incapable of noticing minute frequency changes

Three-Stage Pipeline

The JPEG compression pipeline consists of three main stages that work sequentially:

Color Space Conversion

transforms RGB to YCbCr, DCT + Quantization



DCT + Quantization

transforms spatial data to frequency domain and discards high-frequency components



Entropy Encoding

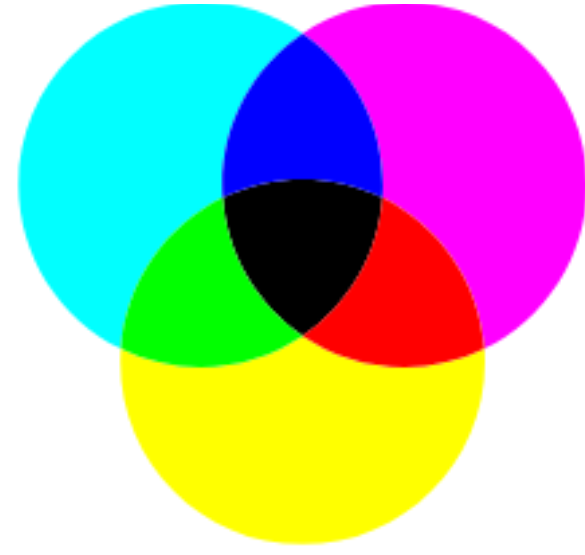
applies Huffman coding for final lossless compression.

Color Spaces

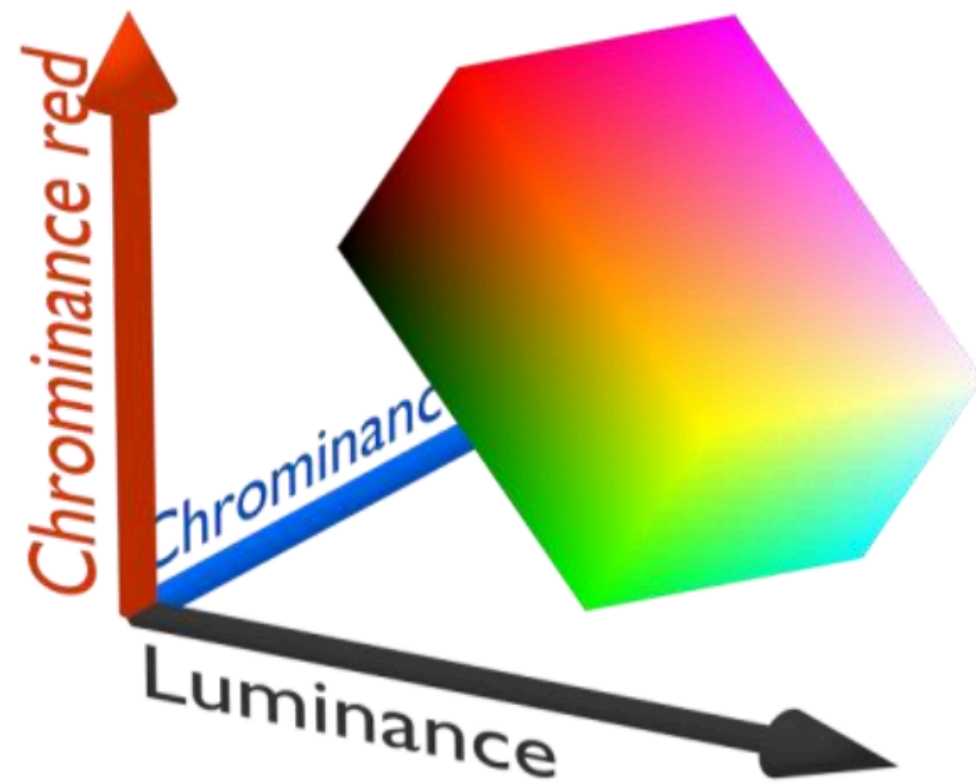
Understanding different color representations is critical for JPEG compression, as the algorithm relies on separating luminance from chrominance information.



RGB



CMYK



YCbCr

RGB: Additive Model



Used by monitors and displays. Adding Red + Green + Blue = white light. Each channel ranges from 0-255 intensity values.

CMYK: Subtractive Model



Used by printers. Adding inks makes the image darker. 'K' (black) exists because mixing Cyan + Magenta + Yellow produces muddy brown, not true black.

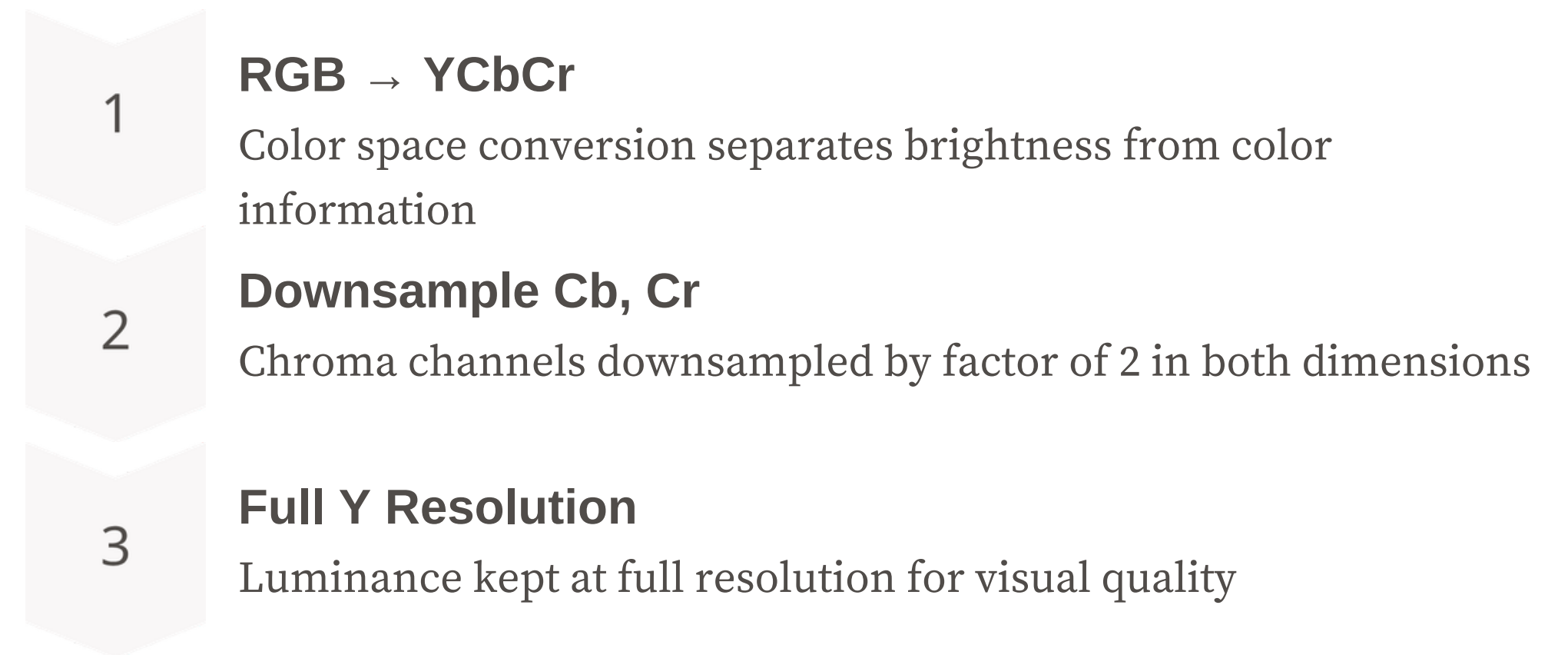
YCbCr: Luminance-Chrominance



Used in TV broadcast and video. Separates luminance (Y = brightness/grayscale) from chrominance (Cb = blueness, Cr = redness). Critical for JPEG chroma downsampling.

Chroma Downsampling

The first step in JPEG compression converts from RGB to YCbCr color space. This transformation is essential because the human eye resolves brightness (luminance) at far higher resolution than color (chrominance).



This step alone reduces color data by **4×** before any DCT is applied, dramatically shrinking file size with minimal perceptual quality loss. Higher quality settings reduce the amount of downsampling applied.

Key Insight

Human vision prioritizes brightness over color detail, making chroma downsampling highly effective



Discrete Cosine Transform

The DCT transforms image data from the spatial domain into the frequency domain, enabling efficient compression by separating important low-frequency information from discardable high-frequency details.

Step 1: Pick an 8×8 block from your image

62	55	55	54	49	48	47	55
62	57	54	52	48	47	48	53
61	60	52	49	48	47	49	54
63	61	60	60	63	65	68	65
67	67	70	74	79	85	91	92
82	95	101	106	114	115	112	117
96	111	115	119	128	128	130	127
109	121	127	133	139	141	140	133

Step 2: Subtract 128 (center around zero)

-66	-73	-74	-79	-80	-81	-73	-75
-66	-71	-74	-76	-80	-81	-79	-74
-67	-68	-76	-79	-80	-81	-60	-63
-65	-67	-68	-68	-65	-63	-37	-36
-61	-61	-58	-54	-49	-43	-16	-11
-46	-33	-27	-22	-14	-13	0	2
-32	-17	-13	-9	0	5	11	13
-19	-7	-1	-1	5	11	12	5

α

Step 3: Coefficient Calculation

DCT computes coefficients representing frequency component magnitudes.

The 8×8 block of pixels becomes an 8×8 block of coefficients. Top-left = DC (average brightness). As you move right/down, frequency increases.

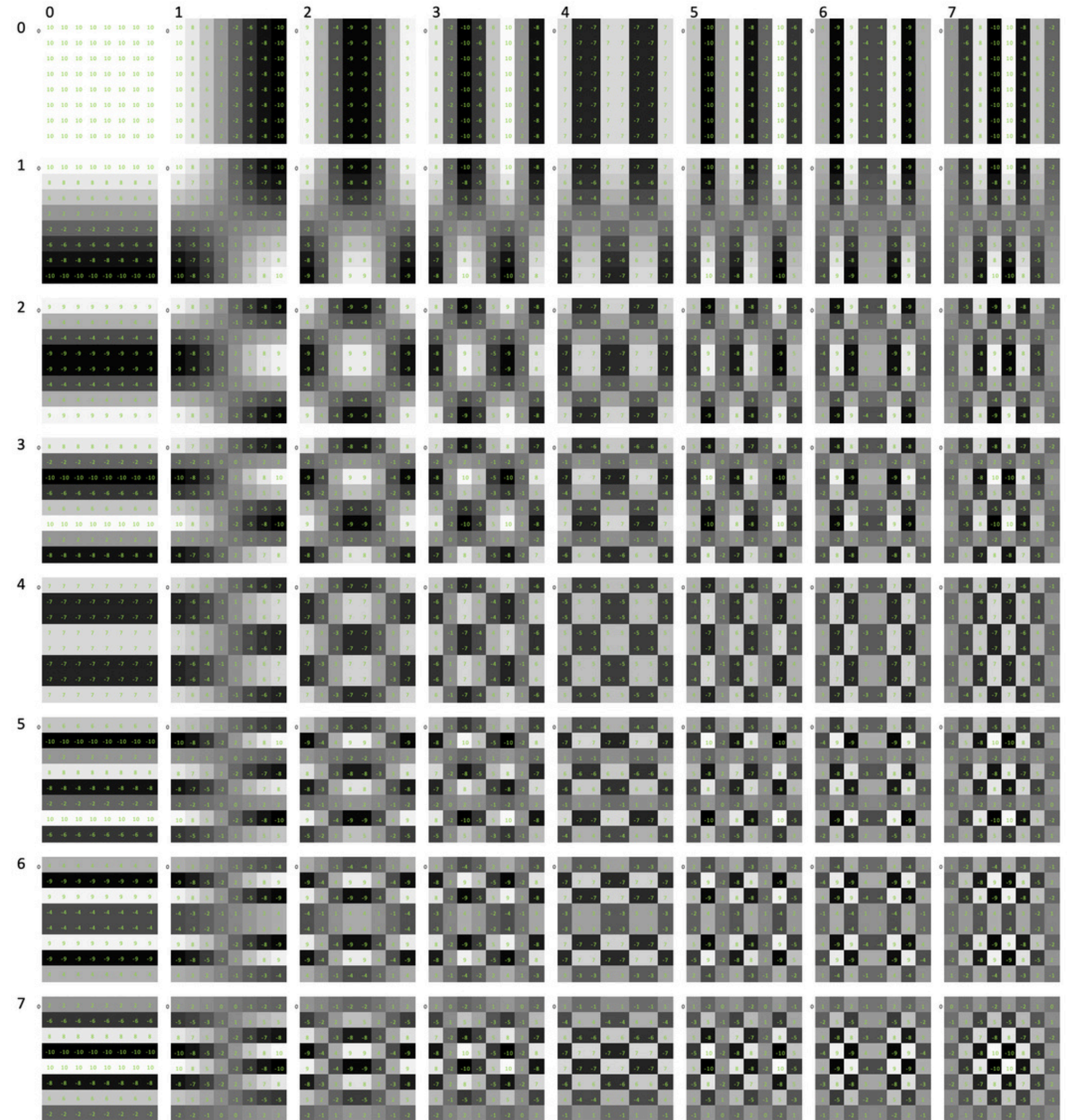
DCTII Coefficients

370	29.7	-2.6	-2.5	-1.1	-3.7	-1.5	0.08
-231	44.9	24.5	-0.3	9.3	3.9	4.3	-1.4
62.8	8.5	-7.6	-2.7	0.3	-0.4	0.5	-0.8
12.5	14.6	-3.5	-3.4	2.4	-1.3	2.7	-0.4
-4.9	-3.9	0.9	3.6	0.1	5.1	1.1	0.5
-0.5	3.1	-1.4	0.2	-1.1	-1.5	-1.1	0.9
4.4	2.3	-1.7	-1.6	1.1	-2.7	1.1	-1.4
-10.2	-1.8	5.9	-0.4	0.3	0.4	-1	0

Low-frequency coefficients (top-left) carry most visual energy and remain intact. High-frequency coefficients (bottom-right) are usually small and often discarded during quantization, representing fine details imperceptible to human vision.

What this shows :

- Each small block is a **DCT basis function** (filter)
- Used to transform image data from spatial domain
→ frequency domain



Quantization & Entropy Encoding

1 — Quantization

Each DCT coefficient divided by quantization table value and rounded to nearest integer

JPEG Quantization Table (Quality = 50)							
16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

2 — Data Loss

High-frequency coefficients rounded to zero—this is where actual data loss occurs

Quantized							
-23	-2	0	0	0	0	0	0
-21	4	2	0	0	0	0	0
6	1	0	0	0	0	0	0
1	-1	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

3 — Zig-Zag Scan

Remaining coefficients serialized in zig-zag pattern to cluster zeros together

4 — Huffman Encoding

Lossless compression applied to coefficient stream for final bit reduction

Decompression Pipeline

- 1. Multiply by quantization table
- 2. Apply Inverse DCT
- 3. Reconstruct pixels

Quality Control

Quality setting controls quantization table values: higher quality = smaller divisors = more coefficients survive

Why Parallelize Image Compression?

JPEG compression represents a classic **data-parallel** workload perfectly suited for GPU acceleration. Each 8×8 block in JPEG is completely independent—no block depends on another for DCT or quantization operations.

Massive Parallelism

A 4K image contains approximately 1,40,000 8×8 blocks, ideal for simultaneous processing

GPU Advantage

Thousands of GPU cores process all blocks simultaneously rather than sequentially

CPU Limitation

Single-threaded CPU processes blocks one at a time, creating sequential bottleneck

~100% Parallelizable Fraction

- Amdahl's Law states that speedup is limited by the sequential portion of a program — even 5% serial work caps your speedup at $20\times$, regardless of core count.
- In JPEG compression, the only serial work is one-time setup: loading the image and uploading DCT/quantization tables to GPU memory. The actual compression across all 8×8 blocks is fully parallel — no thread ever waits on another.
- This makes JPEG one of the best real-world cases for GPU acceleration.



Content-Aware Quality Mapping

Our project extends standard JPEG with **saliency-based quality mapping**, applying variable compression levels across different image regions based on visual importance.

01

Standard JPEG Limitation

Single quality level applied uniformly across entire image

02

Saliency Detection

Compute per-block importance from local contrast and luma variance

03

Quality Scaling

Important regions receive higher quality, background gets more compression

04

Perceptual Gain

Same file size with better quality, or smaller file with same apparent quality

Implementation

Per-block quality scales computed from importance map and applied before quantization step

Result

Content-aware compression improves perceptual quality at reduced bitrates



GPU CUDA Kernel & Performance Results

Thread Block Size 8×8 threads maps exactly to one JPEG block	Grid Dimensions (image_width/8) × (image_height/8) blocks processes every block in parallel
Shared Memory 8×9 float arrays (+1 padding avoids bank conflicts)	Constant Memory DCT matrix (T), transpose (Tt), quantization table (d_qLuma) uploaded once, broadcast to all threads

GPU vs CPU — Benchmark Results

Metric	GPU	CPU
Compression time (test image)	22.625 ms	138.928 ms
Speedup	6.140x	1x
Output quality (PSNR)	68.361	-
Quality map support	Yes	Yes

Both paths produce pixel-identical output verified via PSNR comparison. GPU advantage grows with image resolution as more blocks enable greater parallelism. Content-aware mode achieves same or better perceptual quality at reduced bitrate.





Thank
you!